Product Requirements Document (PRD): AuraFlow

Product Name: FluxHeal — Autonomous Self-Healing ETL/ELT Data Pipeline

Target Release: Clean-Tech Hackathon 2026 MVP (v1.0) & Enterprise Roadmap

Tech Stack: NestJS (TypeScript), PostgreSQL (Relational + `JSONB`), Socket.io, React + Tailwind CSS

Status: Approved for Engineering Execution

---

1. Product Overview & Strategic Alignment

1.1 Product Purpose

AuraFlow is an autonomous, zero-downtime data engineering platform that detects, isolates, diagnoses, and repairs broken ETL (Extract-Transform-Load) and ELT (Extract-Load-Transform) data pipelines in real time. Built specifically for data-intensive Clean-Tech infrastructures—such as solar/wind smart grids, precision agriculture IoT networks, EV charging fleets, and ESG carbon-accounting systems—AuraFlow eliminates pipeline downtime while significantly reducing cloud compute energy waste.

1.2 Alignment with Clean-Tech Hackathon Themes

AuraFlow directly addresses core Clean-Tech impact areas by improving resource and energy efficiency, reducing emissions and compute waste, and enabling uninterrupted sustainable operations:

1. **Green IT & Scope 3 Cloud Emissions Reduction:** Traditional pipelines crash on minor data errors, forcing engineers to run massive historical backfills that waste server CPU cycles and data-center electricity. AuraFlow heals records incrementally in flight, cutting redundant cloud compute energy by up to 40%.
2. **24/7 Operational Visibility for Climate Infrastructure:** Renewable grids and automated irrigation systems rely on continuous sensor telemetry. AuraFlow ensures corrupt sensor packets or firmware schema updates never blind real-time sustainability operations.
3. **Audit-Ready ESG & Carbon Data Unification:** Automatically normalizes fragmented, multi-vendor environmental data while preserving full cryptographic-grade lineage for regulatory audits.

---

2. Target User Personas

Persona	Role & Environment	Core Pain Points	How AuraFlow Solves It
1. The Clean-Tech Data / IoT Engineer (*Primary Builder*)	Manages 1,000+ solar inverters, smart meters, or agricultural IoT sensors streaming into PostgreSQL.	Wakes up at 2 AM because a firmware update renamed a JSON key (`solar_kw` ➔ `active_pwr_kw`) or a faulty sensor sent `-9999` , crashing the ingestion worker.	Isolates the faulty packet in `<1ms` , auto-maps the schema drift or imputes the sensor spike, caches the fix, and keeps the pipeline at 100% uptime.
2. The ESG & Sustainability Analyst (*Business / Compliance User*)	Aggregates energy bills, grid emissions, and factory telemetry for carbon reporting.	Missing data points, inconsistent unit strings (`"230.5 V"` vs `230.5`), and failed nightly SQL reports delay compliance audits.	Auto-sanitizes unit formats, patches broken warehouse SQL queries via `SAVEPOINT` sandboxes, and attaches an immutable `_healed_meta` audit trail.
3. The Hackathon Judge / Technical Executive (*Evaluator*)	Evaluates technical depth, live product viability, and measurable Clean-Tech impact within a 3–5 minute demo.	Backend infrastructure tools often look boring or static during live pitches.	Provides an interactive **Chaos Fault Injector Dock** and a **60fps Visual Node Canvas** where judges can break the pipeline live and watch it self-heal in seconds.

---

3. Product Goals & Success Metrics (KPIs)

Metric Category	Key Performance Indicator (KPI)	Traditional Pipeline Baseline	AuraFlow v1.0 Target
Reliability	Pipeline Uptime During Schema/Data Faults	Drops to `0%` (Halts on unhandled exception)	`99.99%` **(Zero-Downtime Isolation)**
Speed (MTTR)	Mean Time to Resolution (Detection	`2 – 6 Hours` (Manual engineer	`< 1.5 Seconds` **(AI Diagnosis) /** `< 1ms` **(Cached**

Metric Category	Key Performance Indicator (KPI)	Traditional Pipeline Baseline	AuraFlow v1.0 Target
	to Fix)	debugging)	**Rule)**
Performance	Fast-Path Ingestion Overhead	N/A	`< 2ms` **validation + hot-cache latency**
Sustainability	Redundant Backfill Rows Avoided	`100%` of batch re-processed on failure	`99.9%+` **of healthy batch rows skipped from re-runs**
Accuracy	Autonomous Healing Precision	N/A	`≥ 95%` **on deterministic & semantic fixes (with HITL fallback)**

---

4. Functional Requirements & Feature Specifications (MoSCoW Prioritized)

Epic 1: High-Throughput Ingestion & Hot-Rule Cache (`NestJS IngestionModule`)

ID	Priority	Requirement Description	Technical / Product Behavior
FR-1.1	**Must Have**	**Multi-Source Stream & Batch Ingestion**	Expose REST (`POST /api/v1/ingest/:connector`) and WebSocket ingestion endpoints capable of receiving single JSON telemetry packets or bulk array batches.
FR-1.2	**Must Have**	**Sub-Millisecond Hot-Rule Cache**	Before validation, compute a structural hash (`source_signature`) of incoming payload keys/types and check `HotRuleCacheService` (`Map<string, Rule>`). If a learned rule exists, apply the transformation in memory in `<1ms` .
FR-1.3	**Must Have**	**Schema, Type & Physical Boundary Gate**	Validate payloads against target schema contracts (required fields, data types) and physical Clean-Tech boundaries (e.g., `temperature_c` between `-50` and `85` , `moisture_pct` between `0` and `100`).

Epic 2: Zero-Downtime Micro-Quarantine (`NestJS QuarantineModule`)

ID	Priority	Requirement Description	Technical / Product Behavior
FR-2.1	**Must Have**	**Row-Level Fault Isolation**	If a batch of 1,000 records contains 2 invalid records, write the 998 valid records immediately to `clean_telemetry_warehouse` and route only the 2 defective payloads to `quarantine_dlq`.
FR-2.2	**Must Have**	**Asynchronous Event Decoupling**	Emit `@OnEvent('pipeline.anomaly.detected')` via NestJS `EventEmitter2` so quarantine logging and AI diagnosis never block the primary ingestion thread.
FR-2.3	**Should Have**	**Contextual Snapshot Capture**	Store the raw broken JSON payload, connector ID, timestamp, exact validation error stack, and the last 5 historical values for that `device_id` inside the quarantine record.

Epic 3: Dual-Mode Autonomous Healing Engine (`NestJS HealerModule` & `EltWarehouseModule`)

ID	Priority	Requirement Description	Technical / Product Behavior
FR-3.1	**Must Have**	**Semantic Schema-Drift Resolution (ETL)**	When unknown keys arrive (e.g., `active_pwr_kw` instead of `power_kw`), use fuzzy string matching + LLM semantic analysis + data-type inspection to synthesize a key-mapping rule automatically.
FR-3.2	**Must Have**	**Time-Series Sensor Imputation (ETL)**	When an IoT sensor reports `null`, `NaN`, or an out-of-bounds spike (`-9999°C`), `TimeSeriesWindowService` calculates the Exponential Moving Average (EMA) from the last 20 valid readings of that sensor and imputes a realistic value.
FR-3.3	**Must Have**	**Format & Unit String Sanitizer**	Automatically strip unit suffixes from numeric strings (e.g., `"64.8 kW"` ➔ `64.8`, `"230.5 V"` ➔ `230.5`) and

ID	Priority	Requirement Description	Technical / Product Behavior
		(ETL)	normalize heterogeneous date strings into standard ISO-8601 `TIMESTAMPTZ` .
FR-3.4	**Must Have**	**Transactional ELT SQL Patching (`SAVEPOINT`)**	Intercept PostgreSQL runtime errors (`42703` undefined column, `22012` division by zero, `22P02` invalid cast). Rollback to a transaction `SAVEPOINT` , rewrite the SQL query (e.g., adding `NULLIF` or `COALESCE`), verify in a dry-run savepoint, and commit.
FR-3.5	**Must Have**	**Self-Learning Rule Compilation**	Once a healing rule succeeds with confidence ≥ 0.90, persist it to `healing_rules_registry` and push it to `HotRuleCacheService` so all subsequent identical errors are healed deterministically without calling the LLM.

Epic 4: Green Compute & Carbon Accounting (`NestJS GreenMetricsModule`)

ID	Priority	Requirement Description	Technical / Product Behavior
FR-4.1	**Must Have**	**Avoided Backfill & CPU Calculator**	For every healed anomaly, calculate the number of healthy batch/table rows that did *not* need to be re-processed, converting avoided compute time into **CPU Hours Saved** and **Energy Saved (Wh)**.
FR-4.2	**Must Have**	**Real-Time Carbon Savings Ledger (gCO_2e)**	Multiply saved kWh by regional grid carbon intensity (gCO_2/kWh) and log every saving event in `green_compute_ledger` . Stream live updates to the UI top bar.
FR-4.3	**Could Have**	**Carbon-Aware ELT Batch Deferral**	Display a scheduler toggle that defers non-urgent heavy ELT warehouse aggregations to low-carbon grid windows (when solar/wind generation is peaking).

Epic 5: Governance, Lineage & Human-in-the-Loop (HITL)

ID	Priority	Requirement Description	Technical / Product Behavior
FR-5.1	**Must Have**	**Immutable `_healed_meta` Lineage Tagging**	Every healed record inserted into `clean_telemetry_warehouse` must set `is_healed = TRUE` and populate `_healed_meta` with `{ raw_value, anomaly_type, rule_id, confidence, healed_at }`.
FR-5.2	**Must Have**	**Confidence-Gated HITL Approval**	Fixes with confidence ≥ 0.90 auto-deploy. Fixes with confidence < 0.90 remain in `PENDING_HITL` status and display a **1-Click Approve / Reject** card in the UI.
FR-5.3	**Should Have**	**One-Click Rule Rollback**	Operators can click `[Revert Rule]` on any auto-applied rule to disable it in `HotRuleCacheService` and restore affected warehouse rows from `_healed_meta.raw_value`.

Epic 6: Interactive Mission Control UI & Chaos Simulator (`React + Canvas + Socket.io`)

ID	Priority	Requirement Description	Technical / Product Behavior
FR-6.1	**Must Have**	**60fps Node Topology Canvas**	Render an interactive visual graph (`Data Source` ➔ `Validator Gate` ➔ `Quarantine DLQ` ➔ `AI Healer` ➔ `PostgreSQL Warehouse`) with color-coded animated particles driven by WebSocket events.
FR-6.2	**Must Have**	**Chaos Fault Injector Dock**	Provide 4 dedicated 1-click simulation buttons on the main dashboard: `[⚡ Schema Drift]`, `[🌡️ Sensor -9999 Spike]`, `[🔤 Dirty Units]`, and `[💥 Break ELT SQL]`.
FR-6.3	**Must Have**	**Live Side-by-Side Diff Inspector**	Display the broken raw payload (Red) beside the healed output payload (Green), the synthesized transformation rule/SQL diff, and confidence score in real time.

---

5. Non-Functional Requirements (NFRs)

1. **Latency & Throughput:**

- Fast-Path validation and cached-rule execution must complete in **$< 2\text{ms}$ per payload** at the NestJS service layer.
- WebSocket UI state broadcasts must render within **$< 50\text{ms}$** of backend state transitions.

2. **Zero Data Loss Guarantee:**

- No incoming payload may ever be dropped silently. Every record must either land in `clean_telemetry_warehouse` or `quarantine_dlq` with full ACID transactional durability in PostgreSQL.

3. **LLM Cost & Privacy Guardrails:**

- The AI Healer must **never** send bulk dataset rows to the LLM API. Only schema headers, error codes, and a single anonymized sample record may be transmitted, and once a rule is cached, LLM calls for that signature drop to **zero**.

4. **Resilience:**

- If the external LLM API times out or is unreachable, the Tier-1 Deterministic Engine (Regex unit stripping, Z-score/EMA sensor imputation, and Levenshtein column matching) must continue operating autonomously with zero degradation.

---

6. Acceptance Criteria & End-to-End Test Scenarios (Gherkin Spec)

Scenario 1: Autonomous Schema Drift Resolution & Caching

- **GIVEN** the pipeline expects `{ device_id, timestamp, power_kw }` from connector `solar-grid-01`
- **WHEN** the user clicks `[⚡ Inject Schema Drift]` sending `{ device_id: "INV-12", timestamp: "...", active_power_kw: 48.5 }`
- **THEN** the `ValidatorService` isolates the packet into `quarantine_dlq` without halting healthy stream packets
- **AND** the `HealerService` maps `active_power_kw` ➔ `power_kw` with confidence ≥ 0.90, merges the healed row into `clean_telemetry_warehouse`, and caches the rule in `HotRuleCacheService`

- **AND** the next packet containing `active_power_kw` bypasses Quarantine and flows directly through the Fast-Path in $<1\text{ms}$.

Scenario 2: IoT Sensor Outlier Imputation

- **GIVEN** sensor `AGRI-MOIST-42` has historical temperature readings averaging `36.5°C` in `TimeSeriesWindowService`
- **WHEN** the user clicks `[🌡️ Sensor -9999 Spike]` sending `{ device_id: "AGRI-MOIST-42", temperature_c: -9999 }`
- **THEN** the physical boundary check flags `-9999` as a `SENSOR_OUTLIER`
- **AND** the healer replaces `-9999` with the moving-average estimate (`~36.5°C`) and writes `{ "raw_value": -9999, "method": "EMA_IMPUTATION" }` into `_healed_meta` .

Scenario 3: Transactional ELT Warehouse SQL Self-Patching

- **GIVEN** a batch of 50,000 smart-meter records in `raw_staging_payloads` where 1 record has `power_consumed = 0`
- **WHEN** the user clicks `[💥 Break ELT SQL]` executing an efficiency calculation (`emissions / power_consumed`)
- **THEN** PostgreSQL raises `SQLSTATE 22012 (division_by_zero)`
- **AND** `EltWarehouseService` rolls back to `SAVEPOINT elt_execution` , rewrites the SQL expression to `emissions / NULLIF(power_consumed, 0)` , commits the transaction, and increments the **Green Compute & CO2 Saved** counters for avoiding a 50,000-row re-run.

---

7. Release Scope & Phased Roadmap

Phase 1: Hackathon MVP Scope (Current Build)

- Complete NestJS modular backend (`Ingestion` , `Validator` , `Quarantine` , `Healer` , `EltWarehouse` , `GreenMetrics` , `ChaosSimulator` , `WebSocketGateway`).
- Complete PostgreSQL schema with `JSONB` audit lineage and `SAVEPOINT` SQL self-patching.
- Single-Pane-of-Glass Mission Control UI with 60fps Node Topology Canvas, Live JSON/SQL Diff Inspector, Green Compute Area Chart, and 4 interactive Chaos Fault Injection buttons.

Phase 2: Post-Hackathon Enterprise V2

- Native Kafka, MQTT, and Apache Iceberg connectors.

- Live Electricity Maps / WattTime API integration for dynamic multi-region carbon-aware workload shifting.
- Slack, Microsoft Teams, and Telegram Bot webhook integrations for interactive mobile HITL approvals.